

3x3s1 Winograd 优化

1. Winograd 介绍

以 $F(2, 3)$ 为例, 计算卷积 $[d_0, d_1, d_2, d_3] * [g_0, g_1, g_2] = [y_0, y_1], \text{stride}=1$:

$$F(2, 3) = \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} d_0 g_0 + d_1 g_1 + d_2 g_2 \\ d_1 g_0 + d_2 g_1 + d_3 g_2 \end{bmatrix}$$

其中,

d_0, d_1, d_2 和 d_1, d_2, d_3 为连续的两个输入序列;

g_0, g_1, g_2 为 FIR 的三个参数;

这个过程一共需要 6 次乘法, 和 4 次加法

而 Winograd 算法指出, $F(2, 3)$ 可以这样计算:

$$F(2, 3) = \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{bmatrix}$$

其中,

$$\begin{aligned} m_1 &= (d_0 - d_2) g_0 & m_2 &= (d_1 + d_2) \frac{g_0 + g_1 + g_2}{2} \\ m_4 &= (d_1 - d_3) g_2 & m_3 &= (d_2 - d_1) \frac{g_0 - g_1 + g_2}{2} \end{aligned}$$

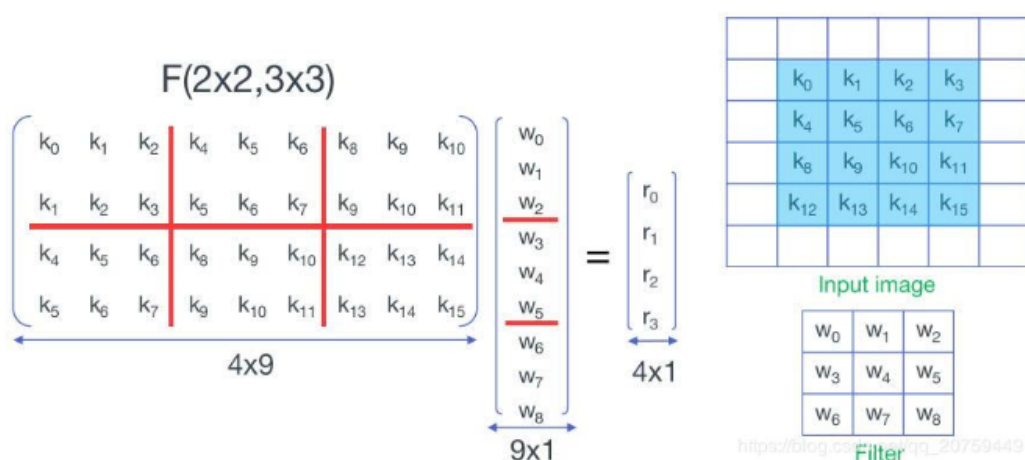
该用矩阵运算可以表示成:

$$Y = A^T [(Gg) \odot (B^T d)]$$

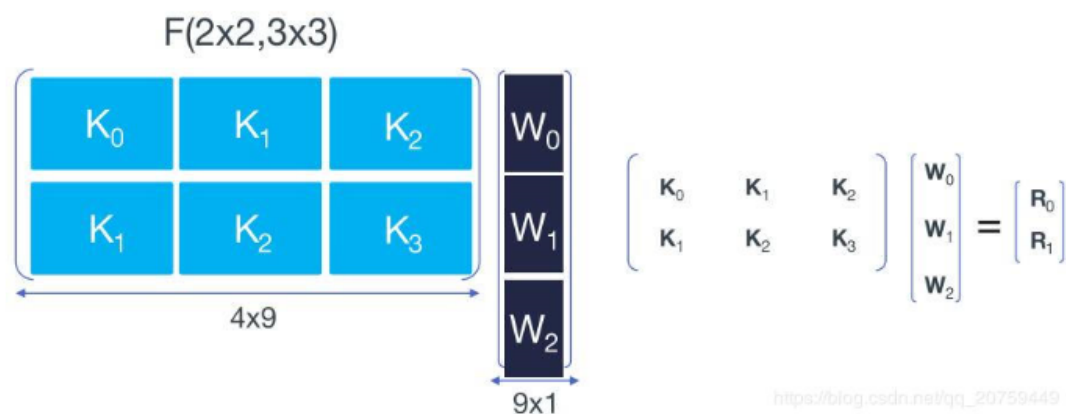
其中, $\odot$ 表示点乘, 而

$$B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}, G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix}, A^T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix}$$

F(2x2, 3x3) 可以由 F(2, 3) 嵌套得到



对矩阵和向量进行分块——



二维 winograd 公式

$$\begin{aligned}
 \begin{bmatrix} K_0 & K_1 & K_2 \\ K_1 & K_2 & K_3 \end{bmatrix} &= \begin{bmatrix} R_0 \\ R_1 \end{bmatrix} = \begin{bmatrix} K_0 W_0 + K_1 W_1 + K_2 W_2 \\ K_1 W_0 + K_2 W_1 + K_3 W_2 \end{bmatrix} \\
 &= \begin{bmatrix} F_{(2,3)}(D_0, W_0) + F_{(2,3)}(D_1, W_1) + F_{(2,3)}(D_2, W_2) \\ F_{(2,3)}(D_1, W_0) + F_{(2,3)}(D_2, W_1) + F_{(2,3)}(D_3, W_2) \end{bmatrix} \\
 &= \begin{bmatrix} A^T [(GW_0) \odot (B^T D_0)] + A^T [(GW_1) \odot (B^T D_1)] + A^T [(GW_2) \odot (B^T D_2)] \\ A^T [(GW_0) \odot (B^T D_1)] + A^T [(GW_1) \odot (B^T D_2)] + A^T [(GW_2) \odot (B^T D_3)] \end{bmatrix} \\
 &= A^T [[G[W_0 \ W_1 \ W_2]G^T] \odot [B^T[D_0 \ D_1 \ D_2 \ D_3]B]] A \\
 &= A^T [[GwG^T] \odot [B^TdB]] A \\
 &(\dots w \Rightarrow g \dots) \\
 &= A^T [[GgG^T] \odot [B^TdB]] A
 \end{aligned}$$

上面公式是 $ic=oc=1$ 的情况，对于实际的卷积，每一个 oc ，需要在 ic 的方向做

累加，可以用 `gemm` 实现所有 oc 的计算。

2. 3x3s1 实现[lite 中只有 f(6,3) 实现，此处以 f(2,3) 为例]

三步走: `input trans(BTDB) → GEMM → output trans(ATOA)`

Weight: $[OC, IC, 3, 3] \rightarrow [OC, IC, 4, 4] \rightarrow [16, OC, IC]$ (预处理)

Input: $[IC, IH, IW]$

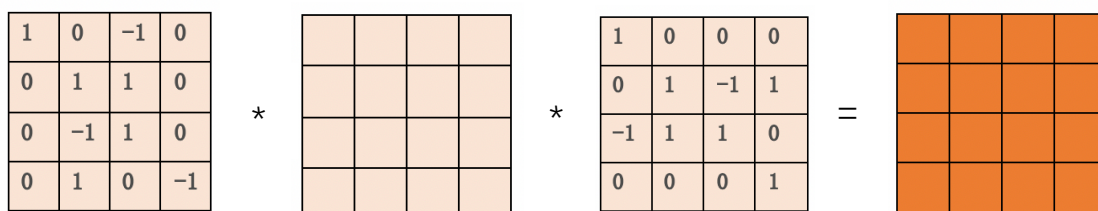
Output: $[OC, OH, OW]$

Input trans

$TileCount = OH/2 * OW/2$

输入 $\rightarrow$ Input: $[IC, TileCount, 4, 4]$

For $TileCount$ (omp): 4x4 Trans(BTDB)

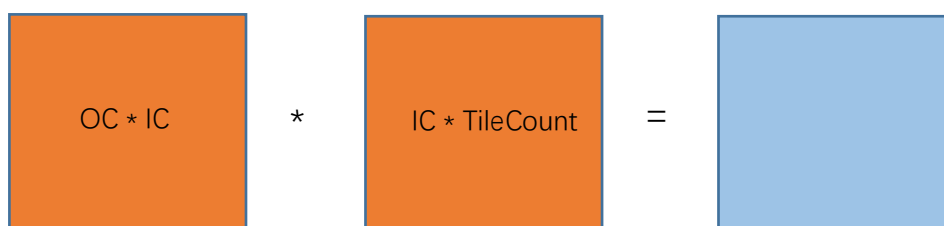


Transpose $[IC, TileCount, 16] \rightarrow [16, IC, TileCount]$

Gemm:

For 0-16 (omp)

GEMM: $[OC, IC] * [IC, TileCount] \rightarrow [OC, TileCount]$

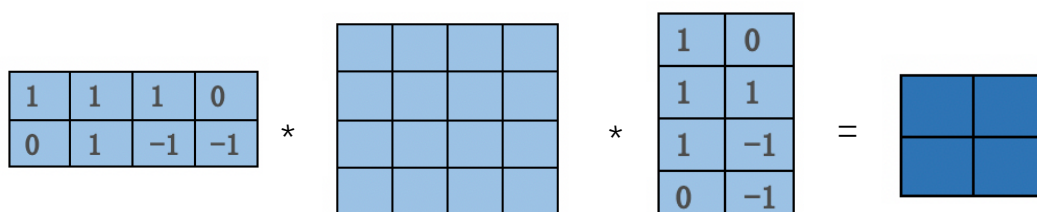


Output trans:

Transpose $[16, OC, TileCount] \rightarrow [OC, TileCount, 16]$

For $TileCount$ (omp)

Trans ATOA $[4, 4] \rightarrow [2, 2]$



输出→Output [OC, OH, OW]

3. 3x3s1 优化 [lite 中有 f(6,3)和 f(2,3)的实现, 此处以 f(2,3)为例]

- 方法: 1. 分块计算, 提高 cache 利用率; 分块数量为 8;
2. 将输入转成 c4 计算, 计算向量化

具体实现:

(预处理) Weight trans [OC, IC, KH, KW] →

[OC/4, IC/4, KH, KW, 4(i), 4(o)] → [16, OC/4, IC/4, 4(i), 4(o)]

TileCount=OH/2 * OW/2, tile_block=8, block_count=TileCount/8

1. Input [IC,H,W]→[IC/4,H,W,4]

2. For block_count (omp)

2-1: Input_trans:

For tile_block: [IC/4,4(h),4(w),4(c)]→[IC/4,4(h),4(w),4(c)]

[IC/4,8,4(h),4(w),4(c)]→[16,IC/4,8,4(c)] (向量化)

C1	C2	C3	C4	C1	C2	C3	C4	C1	C2	C3	C4	C1	C2	C3	C4

*

1	1	1	1	0	0	0	0	-1	-1	-1	-1	0	0	0	0
0	0	0	0	1	1	1	1	1	1	1	1	0	0	0	0
0	0	0	0	-1	-1	-1	-1	1	1	1	1	0	0	0	0
0	0	0	0	1	1	1	1	0	0	0	0	-1	-1	-1	-1

```

float32x4_t src00 = vld1q_f32(src);
float32x4_t src01 = vld1q_f32(src + src_stride);
float32x4_t src02 = vld1q_f32(src + src_stride + src_stride);
float32x4_t src03 = vld1q_f32(src + src_stride + src_stride + src_stride);
src += src_h_stride;
float32x4_t src10 = vld1q_f32(src);
float32x4_t src11 = vld1q_f32(src + src_stride);
float32x4_t src12 = vld1q_f32(src + src_stride + src_stride);
float32x4_t src13 = vld1q_f32(src + src_stride + src_stride + src_stride);
src += src_h_stride;
float32x4_t src20 = vld1q_f32(src);
float32x4_t src21 = vld1q_f32(src + src_stride);
float32x4_t src22 = vld1q_f32(src + src_stride + src_stride);
float32x4_t src23 = vld1q_f32(src + src_stride + src_stride + src_stride);
src += src_h_stride;
float32x4_t src30 = vld1q_f32(src);
float32x4_t src31 = vld1q_f32(src + src_stride);
float32x4_t src32 = vld1q_f32(src + src_stride + src_stride);
float32x4_t src33 = vld1q_f32(src + src_stride + src_stride + src_stride);

float32x4_t dst00 = vsubq_f32(src00, src02);
float32x4_t dst10 = vaddq_f32(src01, src02);
float32x4_t dst20 = vsubq_f32(src02, src01);
float32x4_t dst30 = vsubq_f32(src01, src03);

float32x4_t dst01 = vsubq_f32(src10, src12);
float32x4_t dst11 = vaddq_f32(src11, src12);
float32x4_t dst21 = vsubq_f32(src12, src11);
float32x4_t dst31 = vsubq_f32(src11, src13);

float32x4_t dst02 = vsubq_f32(src20, src22);
float32x4_t dst12 = vaddq_f32(src21, src22);
float32x4_t dst22 = vsubq_f32(src22, src21);
float32x4_t dst32 = vsubq_f32(src21, src23);

float32x4_t dst03 = vsubq_f32(src30, src32);
float32x4_t dst13 = vaddq_f32(src31, src32);
float32x4_t dst23 = vsubq_f32(src32, src31);
float32x4_t dst33 = vsubq_f32(src31, src33);

float32x4_t dest00 = vsubq_f32(dst00, dst02);
float32x4_t dest10 = vaddq_f32(dst01, dst02);
float32x4_t dest20 = vsubq_f32(dst02, dst01);
float32x4_t dest30 = vsubq_f32(dst01, dst03);

float32x4_t dest01 = vsubq_f32(dst10, dst12);
float32x4_t dest11 = vaddq_f32(dst11, dst12);
float32x4_t dest21 = vsubq_f32(dst12, dst11);
float32x4_t dest31 = vsubq_f32(dst11, dst13);

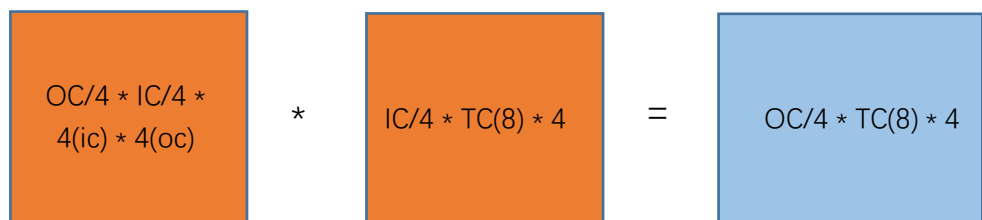
float32x4_t dest02 = vsubq_f32(dst20, dst22);
float32x4_t dest12 = vaddq_f32(dst21, dst22);
float32x4_t dest22 = vsubq_f32(dst22, dst21);
float32x4_t dest32 = vsubq_f32(dst21, dst23);

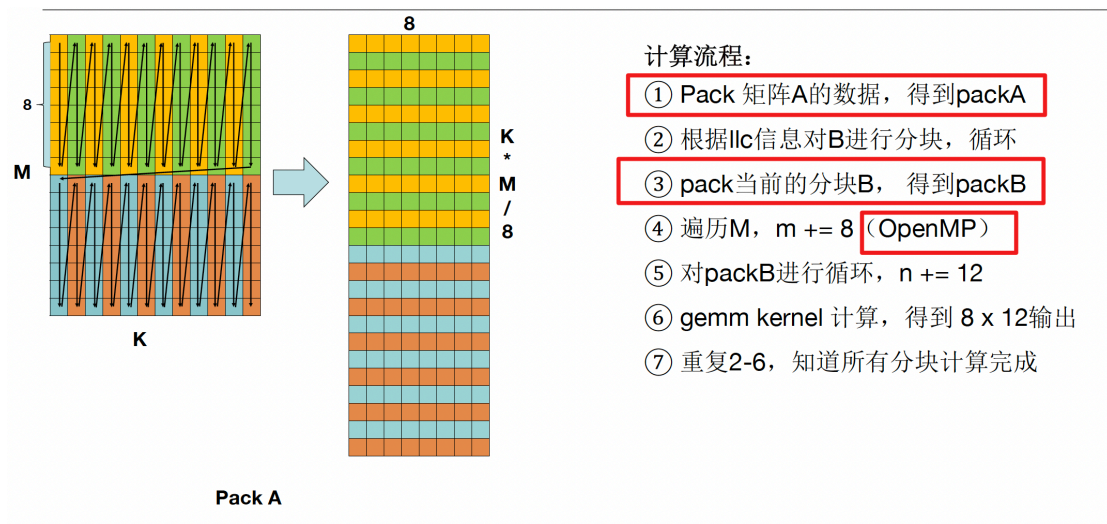
float32x4_t dest03 = vsubq_f32(dst30, dst32);
float32x4_t dest13 = vaddq_f32(dst31, dst32);
float32x4_t dest23 = vsubq_f32(dst32, dst31);
float32x4_t dest33 = vsubq_f32(dst31, dst33);

```

2-2: GEMM(c4 Gemm):

For 0-16: $[OC/4, IC/4, 4, 4] * [IC/4, 8, 4] \rightarrow [OC/4, 8, 4]$





C4 Gemm 相对于普通 Gemm 来说少了对 a 和 b 的预处理

2-3: Output Trans:

(同 input trans)

For tile_block: $[16, OC/4, 4] \rightarrow [OC/4, 2, 2, 4]$

$[16, OC/4, 8, 4] \rightarrow [OC/4, 8, 2, 2, 4]$

3. Output Trans:

$[OC/4, OH, OW, 4] \rightarrow [OC, OH, OW]$

【f(2,3)和f(6,3)的选择】

在 f(6,3), f(2,3) 里 f(6,3) 加速比最高 (约 5 倍), 但如果 oh, ow 太小, 在多线程情况下没法并行, 所以需要根据 oh, ow, threads 选择合适的 kernel, lite 中采用 $U = oh * ow / \text{tile_block}(8) / \text{threads}$ 判断, 如果 $U < 36$, 刚选择 f(2,3), 否则选择 f(6,3); 在选择 f(2,3) 时, 仍有情况不能并行, 此时会选择 f(2,3) 的 small 版本, 该版本中在 block_count 处的 omp 去掉, 加到了 Gemm 的 for(0-16) 上, 以尽可能实现并行化, f(2,3) 的 small 的判断使用公式 $\text{block_count} = oh/2 * ow/2 / \text{tile_block}(8)$ 实现, 如果 $\text{block_count} = 1$, 则选用 small 版本。

【性能】

A73@2.36G	2 中实现	3 中实现 (优化版本)
输入 1x16x64x64, oc=16	14.05G (峰值 74%)	20.78G (峰值 1.1 倍)
输入 1x256x64x64, oc=256	35.65G (峰值 1.88 倍)	41.09G (峰值 2.18 倍)

4. Winograd int8 实现 (目前只有 f(2,3) 实现)

实现思路: 实现思路流程与 float 一致, 只是将输入由 c4 改成 c8

input: int8, weights: int8

Input trans(BTDB): int8 $\rightarrow$ int16

Weights trans(GTgG): int8 $\rightarrow$ int16

由于计算过程中需要使用 int8, 需要将 B, G 系数矩阵全转成整数, 最后计算结果再除以缩放系数, 只有 f(2,3) 中的 g 的转换不存在溢出问题, 故

目前只能实现 f(2,3)

```
(预处理) Weight trans [OC, IC, KH, KW](int8)→  
[OC/8, IC/8, KH, KW, 8(i), 8(o)](int8) →  
[16, OC/8, IC/8, 8(i), 8(o)] (int16)  
TileCount=OH/2 * OW/2, tile_block=8, block_count=TileCount/8  
  
1. Input [IC,H,W]→[IC/8,H,W,8]  
2. For block_count (omp)  
  2-1: Input_trans:  
    For tile_block: [IC/8,4(h),4(w),8(c)](int8) →  
                    [IC/8,4(h),4(w),8(c)](int16)  
                    [IC/8,8,4(h),4(w),8(c)](int8)→[16,IC/8,8,8(c)](int16)  
  2-2: GEMM:  
    For 0-16: [OC/8,IC/8,8,8](int16) * [IC/8,8,8](int16)  
              → [OC/8,8,8](int32)  
  2-3: Output Trans:  
    For tile_block: [16, OC/8, 8] → [OC/8, 2, 2, 8]  
    [16, OC/8, 8, 8] → [OC/8, 8, 2, 2, 8]  
3. Output Trans:  
   [OC/8, OH, OW, 8] → [OC, OH, OW]
```

【注】

Winograd int8 由于 weights 转换时的溢出问题(f(2,3)不存在溢出问题, f(4,3)weights 转换会溢出), 目前只能实现 f(2,3), 而目前 f(2,3)的加速比很难超越 f(6,3), 因此在 int8 情况下大多数情况下仍很难超越 float.